



Módulo I

Lógica de Programação

Estrutura Sequencial em Programação

Nesta Aula

O que vamos aprender hoje?

Esta aula apresenta a **Estrutura Sequencial**, o conceito mais fundamental da programação. Antes de aprender a tomar decisões ou repetir tarefas, precisamos entender como o computador executa instruções em linha reta, uma por vez.

01

O Conceito

O que é estrutura sequencial e como ela funciona?
Entenda como os computadores executam instruções passo a passo — a base de todo algoritmo bem construído.

03

Exemplos Práticos

Algoritmos em pseudocódigo para fixar o aprendizado

02

Entrada, Processamento e Saída

As três fases essenciais de qualquer algoritmo

04

Reflexão Crítica

Limites da estrutura e o que vem a seguir

A Analogia da Receita de Bolo

Imagine que você tem uma receita de bolo. Você não começa misturando os ingredientes antes de separá-los, certo? E jamais coloca o bolo no forno antes de ligá-lo. A receita impõe uma **ordem natural**: primeiro uma coisa, depois outra, até o resultado final estar pronto.

Na programação, a lógica é exatamente a mesma. A **Estrutura Sequencial** é um fluxo de execução onde as instruções são realizadas em ordem linear, uma após a outra, exatamente como foram escritas — sem atalhos, sem desvios.



O que é a Estrutura Sequencial?

Definição

A Estrutura Sequencial é um **fluxo de execução linear** onde cada instrução (comando) é executada na exata ordem em que foi escrita no algoritmo.

É o tipo mais simples e fundamental de estrutura em programação — e é o ponto de partida para entender toda a lógica computacional.

Características Principais

- As instruções são executadas **uma de cada vez**, do início ao fim
- **Não há desvios** ("se isso, faça aquilo")
- **Não há repetições** ("faça isso 10 vezes")
- É um **caminho reto**, do início ao fim do algoritmo
- Toda linguagem de programação começa por aqui

As Três Fases de um Algoritmo Sequencial

Didaticamente, todo algoritmo sequencial pode ser dividido em três partes essenciais que se encadeiam em ordem:

1. Entrada

O programa **recebe** os **dados** necessários para trabalhar. Pode ser via teclado, arquivo, sensor ou outra fonte. É a matéria-prima do algoritmo.



2. Processamento

O "cérebro" da operação. O programa **pega** os **dados de entrada** e realiza os cálculos, manipulações ou transformações necessárias para gerar um resultado.

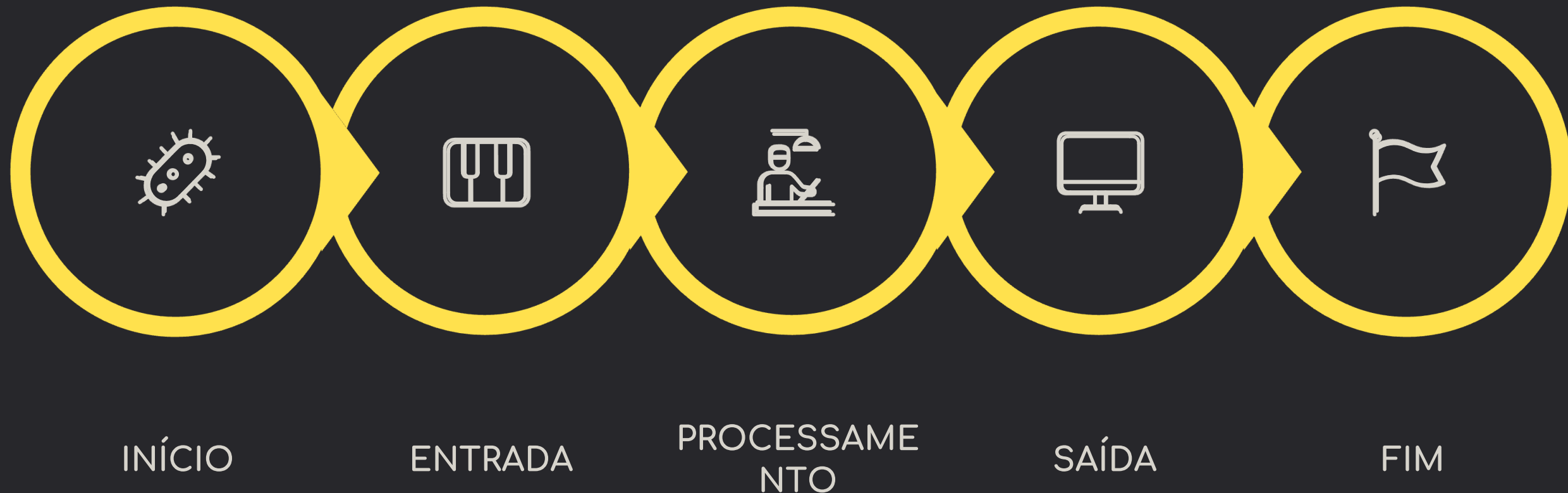


3. Saída

A **devolutiva do programa** para o mundo exterior. O resultado é exibido ao usuário na tela, salvo em arquivo ou enviado para um dispositivo.

Visualizando o Fluxo Sequencial

Observe abaixo o caminho que o programa percorre. Note a **seta única sempre apontando para frente** — não há retorno, não há desvios, apenas uma direção.



Essa linearidade é o que define a estrutura sequencial. Cada bloco depende do anterior e alimenta o próximo — assim como os passos de uma receita ou as etapas de uma linha de montagem.

Exemplo 1: O Assistente Pessoal

Vamos criar um algoritmo em pseudocódigo que funciona como um pequeno assistente. Ele vai perguntar seu **nome** e seu **ano de nascimento**, e depois te dizer, aproximadamente, quantos anos você tem. É um exemplo simples, mas que ilustra perfeitamente as três fases da estrutura sequencial em ação.

PSEUDOCÓDIGO

EXEMPLO PRÁTICO 1



Código: Algoritmo "CalculaIdade"

```
Algoritmo "CalculaIdade"
Var
    nome: caractere
    anoNascimento, anoAtual, idade: inteiro
Inicio
    // --- FASE 1: ENTRADA ---
    Escreva("Olá! Qual é o seu nome? ")
    Leia(nome)
    Escreva("Em que ano você nasceu? ")
    Leia(anoNascimento)
    Escreva("Em que ano estamos? ")
    Leia(anoAtual)

    // --- FASE 2: PROCESSAMENTO ---
    idade <- anoAtual - anoNascimento

    // --- FASE 3: SAÍDA ---
    Escreva("Prazer em te conhecer, ", nome, "!")
    Escreva("Você tem ou fará ", idade, " anos.")
Fimalgoritmo
```


Analizando o "Calculadade" Linha a Linha

1

Entrada — `Leia(...)`

O programa **pausa e espera** o usuário digitar. Os dados inseridos (nome, anoNascimento, anoAtual) são guardados em **variáveis** — "caixinhas" na memória do computador que armazenam valores para uso posterior. É a matéria-prima do algoritmo.

2

Processamento — `idade <- ...`

O computador busca os valores guardados nas variáveis `anoAtual` e `anoNascimento`, realiza a **subtração** e armazena o resultado em uma nova variável chamada `idade`. A "receita" está sendo executada.

3

Saída — `Escreva(...)`

O computador busca o conteúdo das variáveis `nome` e `idade` e **exibe tudo na tela** de forma organizada. É o produto final do nosso algoritmo — o resultado que o usuário vê.

Compreendendo Variáveis

O que são Variáveis?



Pense em variáveis como caixinhas etiquetadas na memória do computador. Cada caixinha tem um nome único (como `nome`, `idade`, `anoNascimento`) e guarda um único valor por vez.

Quando usamos `Leia(anoNascimento)`, estamos dizendo ao computador: *"Guarde o que o usuário digitar dentro da caixinha chamada `anoNascimento`."*

O operador `<-` (seta para esquerda) representa a **atribuição**: coloca um valor dentro de uma variável. Ex: `idade <- 20` significa "guarde 20 na caixinha `idade`".

Exemplo 2: A Calculadora de Troco

Vamos a outro exemplo para fixar o aprendizado. Imagine um programa simples para um vendedor de lanches. O vendedor informa o valor do produto e quanto o cliente pagou — o programa calcula e exibe o troco automaticamente. Mais uma vez, as três fases aparecem de forma clara e precisa.

PSEUDOCÓDIGO

EXEMPLO PRÁTICO 2



Código: Algoritmo "CalculaTroco"

```
Algoritmo "CalculaTroco"
Var
    valorProduto, valorPago, troco: real
Inicio
    // ENTRADA
    Escreva("Qual o valor do produto? R$ ")
    Leia(valorProduto)
    Escreva("Qual o valor pago pelo cliente? R$ ")
    Leia(valorPago)

    // PROCESSAMENTO
    troco <- valorPago - valorProduto

    // SAÍDA
    Escreva("O troco a ser devolvido é de R$ ", troco)
Fimalgoritmo
```

Repare na **clareza e simplicidade** do código. Cada seção tem exatamente uma responsabilidade: receber dados, calcular, exibir. Isso é a elegância da estrutura sequencial.

Comparando os Dois Exemplos

Apesar de resolverem problemas diferentes, os dois algoritmos seguem o **mesmo padrão estrutural**. Veja como as três fases se repetem:

Fase	CalculaIdade	CalculaTroco
Entrada	Leia nome, anoNascimento, anoAtual	Leia valorProduto, valorPago
Processamento	idade ← anoAtual - anoNascimento	troco ← valorPago - valorProduto
Saída	Escreva nome e idade calculada	Escreva o valor do troco

Perceba: o **padrão Entrada → Processamento → Saída** é universal. Independentemente do problema que você precise resolver, essa estrutura se mantém. Identificá-la é o primeiro passo para pensar como programador.



Reflexão Crítica: Os Limites da Estrutura Sequencial

O que aconteceria no "CalculaTroco" se o cliente pagasse um **valor menor** que o do produto? O programa ainda executaria normalmente — mas apresentaria um **troco negativo** (por exemplo, R\$ -5,00), o que não faz sentido no mundo real.

O que a estrutura sequencial faz

Ela simplesmente **executa a sequência** definida: pega `valorPago`, pega `valorProduto`, subtrai um do outro e mostra o resultado. Sem questionar, sem verificar.

O que ela não faz

Ela **não toma decisões**. Para evitar o troco negativo, precisaríamos de uma **estrutura condicional** — que só aprenderemos nas próximas aulas. Cada estrutura tem seu momento!

Resumo e Próximos Passos

Nesta aula, você aprendeu o **fundamento mais importante da programação**. Todo programa mais complexo que você vai escrever no futuro será construído sobre essa base.

Estrutura Sequencial

Instruções executadas em ordem linear, uma após a outra, do início ao fim.

Entrada

O programa recebe dados do mundo exterior por meio de `Leia()`.

Processamento

Os dados são transformados por cálculos e atribuições de variáveis.

Saída

O resultado é apresentado ao usuário via `Escreva()`.

Próxima aula: Estruturas Condicionais — aprenderemos a ensinar o computador a **tomar decisões** com base nos dados, resolvendo problemas como o troco negativo que identificamos hoje.